Práctica Percepción

Aitor Ruiz Bautista
Juan Pérez Fernández

28 de marzo de 2025

1. Introducción

En el ámbito de la robótica, la extracción de información acerca del entorno es fundamental de cara a la realización de cualquier tarea. Este proceso se realiza mediante la abstracción de los datos proporcionados por los diferentes sensores extereoceptivos. La información proporcionada por una cámara es abundante y permite dotar al sistema del conocimiento necesario para interactuar de forma inteligente. Para ello, es necesario la correcta calibración previa de la cámara. En este trabajo se muestra el pipeline para solucionar el problema de calibración en un sistema monocular y se demuestra su efectividad mediante la representación en realidad aumentada de un objeto tridimensional sobre el patrón de calibración.

2. Estado del Arte

La calibración de cámaras es un proceso esencial en visión por computador para garantizar mediciones precisas del entorno. Este proceso permite corregir distorsiones ópticas así como obtener los parámetros intrínsecos y extrínsecos de la cámara. Uno de los métodos más utilizados en la comunidad científica y en aplicaciones industriales es la calibración mediante un patrón de tablero de ajedrez, ampliamente implementado en software como OpenCV [3].

2.1. Fundamentación Teórica

La calibración de cámaras se basa en principios de geometría proyectiva y optimización matemática. Un modelo comúnmente usado es el modelo de cámara de proyección pinhole, el cual se complementa con modelos de distorsión radial y tangencial para corregir aberraciones ópticas.

El problema de calibración se formula como una optimización de parámetros utilizando métodos numéricos como:

- **Método de Levenberg-Marquardt**: Un algoritmo de optimización no lineal que minimiza el error reproyectado entre los puntos del patrón y su proyección en la imagen [1, 2].

- **Método de Zhang (2000)**: Basado en la minimización de errores utilizando múltiples vistas del patrón para calcular los parámetros intrínsecos y extrínsecos [3].

2.2. Aplicaciones y Proyectos Relevantes

Dada su eficacia y simplicidad, la calibración con patrones de tablero de ajedrez se ha utilizado en múltiples áreas de investigación y proyectos de robótica y visión artificial:

- **SLAM y navegación robótica:**
  - Trabajos como el de Davison et al. (2007) en *MonoSLAM* utilizan calibración precisa para estimar la trayectoria de robots en entornos dinámicos [4].
  - En la industria automotriz, los sistemas de percepción para conducción autónoma emplean calibración para mejorar la precisión de sensores visuales.
- **Realidad aumentada y visión 3D:**
  - Estudios como los de Hartley y Zisserman (2004) exploran la calibración para mejorar la precisión en la proyección de objetos virtuales sobre el mundo real [5].
  - Proyectos de realidad aumentada como *ARToolKit* emplean calibración de cámaras para mejorar la estabilidad de las proyecciones tridimensionales [7].
- **Medición y reconstrucción 3D:**
  - En el ámbito biomédico, trabajos como los de Bouguet (2001) han utilizado calibración para mejorar la precisión en sistemas de reconstrucción volumétrica [6].

3. Metodología

El objetivo es estimar los parámetros intrínsecos, los coeficientes de distorsión y los parámetros extrínsecos de la cámara, de manera que se pueda corregir la distorsión y obtener una proyección precisa de los puntos 3D en el plano de la imagen. Para ello se deben seguir los siguientes pasos:

3.1. Captura de Imágenes

Se capturan múltiples imágenes del patrón de ajedrez desde diversas posiciones y orientaciones. Al capturar las imágenes es necesario que el patrón se observe de forma completa en cada imagen.

3.2. Detección de Esquinas en la Imagen

Para cada imagen, se utiliza la función `cv.findChessboardCorners()` de OpenCV con el fin de obtener las esquinas internas del tablero de ajedrez. Esta función identifica las intersecciones entre las líneas del patrón y devuelve un conjunto de coordenadas 2D (en píxeles).

3.3. Definición de Puntos 3D (Object Points)

Dado que el patrón de ajedrez es plano, se asume la coordenada $Z = 0$ para todos los puntos. Se genera un conjunto de puntos 3D que corresponden a las esquinas internas del patrón. Por ejemplo, para un patrón de 7×6 esquinas se define:

$$(0, 0, 0),\ (1, 0, 0),\ (2, 0, 0), \ldots,\ (6, 5, 0)$$

Estos puntos pueden escalarse según las dimensiones reales de cada cuadrado.

3.4. Calibración de la Cámara

Con las correspondencias entre los puntos 3D (definidos en el sistema del patrón) y los puntos 2D (detectados en las imágenes), se procede a la calibración mediante:

`cv.calibrateCamera()`

Esta función estima los siguientes parámetros fundamentales para la modelación de la cámara [3, 8]:

- **La matriz de cámara (parámetros intrínsecos):** Representa la relación entre las coordenadas del mundo real y la imagen proyectada. Se define como:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

  donde f_x y f_y son las longitudes focales en píxeles, y (c_x, c_y) es el centro óptico de la imagen.

- **Coeficientes de distorsión:** Compensan las deformaciones introducidas por la lente, como la distorsión radial y tangencial, que pueden modelarse con parámetros adicionales [9].

- **Parámetros extrínsecos:** Definen la posición y orientación de la cámara en el espacio 3D con respecto al sistema de referencia del patrón de calibración. Se componen de:

  - **Matriz de rotación** R, de dimensiones 3×3, que describe la orientación de la cámara en el espacio.
  - **Vector de traslación** t, de dimensiones 3×1, que indica el desplazamiento de la cámara con respecto al origen del sistema de referencia del patrón.

La transformación completa del sistema de coordenadas del patrón al sistema de coordenadas de la cámara se expresa mediante la matriz de proyección extrínseca:

$$[R \mid t] = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix}$$

donde cada r_{ij} representa un elemento de la matriz de rotación y t_x, t_y, t_z corresponden a los componentes del vector de traslación [10].

La calibración se realiza minimizando el error de reproyección, que mide la diferencia entre los puntos proyectados y los puntos detectados en la imagen. Para ello, se utilizan técnicas de optimización como el algoritmo de Levenberg–Marquardt [11], asegurando una estimación precisa de los parámetros de la cámara.

3.5. Evaluación del Error de Re-proyección

Para validar la calibración, se re-proyectan los puntos 3D utilizando los parámetros obtenidos (con `cv.projectPoints()`) y se compara la ubicación resultante con las coordenadas 2D detectadas. El error de reproyección se calcula como la media de la norma de la diferencia:

$$\text{Error} = \frac{1}{N} \sum_{i=1}^{N} \|\mathbf{p}_i^{\text{detect}} - \mathbf{p}_i^{\text{proyectado}}\|$$

3.6. Almacenamiento de los Parámetros de Calibración

Finalmente, los parámetros obtenidos (matriz de cámara, coeficientes de distorsión, vectores de rotación y traslación) se pueden guardar en archivos (por ejemplo, en formato XML o YAML) para su posterior uso en aplicaciones de reconstrucción 3D, visión por computador o realidad aumentada.

4. Experimentación

Con el fin de demostrar que la calibración de las cámaras es existosa se han realizado diferentes pruebas. En primer lugar, para evaluar la importancia de un dataset amplio y compuesto por imágenes del patrón en diferentes poses, se ha dividido el dataset de partida (15 imágenes) en lotes reducidos. De esta forma se consigue evaluar la evolución del error de reproyección medio para diferentes lotes de imágenes.

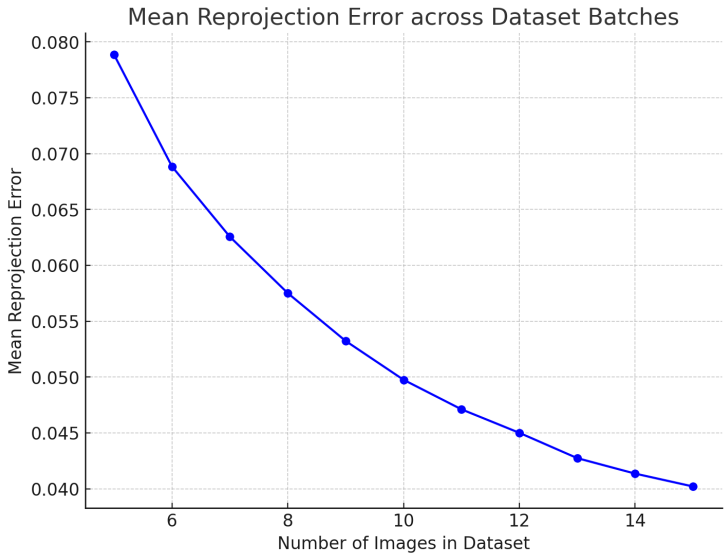


Figura 1: Evolución del Error de Reproyección Medio

En la fig. 1 se observa que la tendencia del error es exponencial y tiende a cero a medida que aumenta el número de imágenes en el dataset. Para evaluar la importancia de tener un error lo menor posible se han mostrado 2 nubes de puntos en poses parecidas con diferentes calibraciones de la cámara.

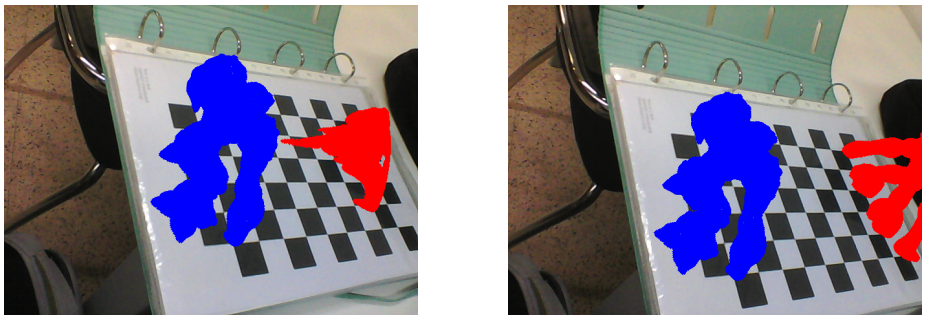


Figura 2: Comparación de proyecciones con diferente calibración

En la fig. 2 se muestra la comparación de proyectar la misma nube de puntos con calibraciones de la cámara diferentes. En la imagen de la izquierda se observa el resultado de proyectar una nube de puntos con un error de reproyección medio cercano al 8 % mientras que la imagen de la derecha muestra la misma nube de puntos con un error aproximado del 4 %. Se demuestra que el error debido a la falta de imágenes del patrón en el dataset incide en que, para ciertas poses de la cámara, se deforme la nube de puntos proyectada. Un error

igual o menor al 5 % resulta en una calibración correcta y robusta ante poses de la cámara que no fueron incluidas en el dataset de calibración.

En [12] se proyectan dos nubes de puntos sobre el patrón de calibración, donde la calibración de cámaras se ha realizado previamente a partir de 15 imágenes con un error de reproyección aproximado al 4 %.

Este GitHub [13] contiene todo el código desarrollado, dataset de imágenes para el patrón de calibración y nubes de puntos empleadas en la demostración.

5. Conclusiones

En robótica el empleo de cámaras es muy común para la estimación de la profundidad en el espacio 3D; casos como la manipulación de objetos mediante efectores finales, en tareas como reconocimiento de objetos y/o escenas. A lo largo de este trabajo vamos a desarrollar un sistema para calibrar una cámara

Los sistemas robóticos a menudo emplean cámaras para el reconocimiento de objetos y escenas, manipulación mediante efectores finales

Referencias

[1] Levenberg, K. (1944). A method for the solution of certain nonlinear problems in least squares. *Quarterly of Applied Mathematics, 2*(2), 164-168.

[2] Marquardt, D. W. (1963). An algorithm for least-squares estimation of nonlinear parameters. *Journal of the Society for Industrial and Applied Mathematics, 11*(2), 431-441.

[3] Zhang, Z. (2000). A flexible new technique for camera calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 22*(11), 1330-1334. https://doi.org/10.1109/34.888730

[4] Davison, A. J., Reid, I. D., Molton, N. D., & Stasse, O. (2007). MonoSLAM: Real-time single camera SLAM. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 29*(6), 1052-1067. https://doi.org/10.1109/TPAMI.2007.1049

[5] Hartley, R., & Zisserman, A. (2004). *Multiple view geometry in computer vision* (2nd ed.). Cambridge University Press.

[6] Bouguet, J.-Y. (2001). *Camera calibration toolbox for Matlab*. Retrieved from `http://www.vision.caltech.edu/bouguetj/calib_doc/`

[7] Kato, H., & Billinghurst, M. (1999). Marker tracking and HMD calibration for a video-based augmented reality conferencing system. *Proceedings of the 2nd IEEE and ACM International Workshop on Augmented Reality (IWAR '99)*, 85-94. https://doi.org/10.1109/IWAR.1999.803191

[8] Heikkilä, J., & Silvén, O. (1997). A four-step camera calibration procedure with implicit image correction. *Proceedings of the IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 1106-1112. https://doi.org/10.1109/CVPR.1997.609429

[9] Brown, D. C. (1966). Decentering distortion of lenses. *Photometric Engineering, 32*(3), 444-462.

[10] Hartley, R., & Zisserman, A. (2003). *Multiple view geometry in computer vision*. Cambridge University Press.

[11] Press, W. H., Teukolsky, S. A., Vetterling, W. T., & Flannery, B. P. (1992). *Numerical recipes in C: The art of scientific computing*. Cambridge University Press.

[12] Demostración de proyección de 2 nubes de puntos sobre patrón de tablero de ajedrez - `https://youtu.be/QUAlUtlV1e0?si=8vyslFySfrPHRqD-`

[13] Práctica 1. Calibración de Cámaras - Aitor R.B. et Juan P.F. - `https://github.com/JuanPerez07/Perception`